

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 02 -

LOAD BALANCERS NA PRÁTICA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	9
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS	12

O QUE VEM POR AÍ?

Na aula anterior, estabelecemos a base da escalabilidade e deixamos uma pergunta crucial no ar: “Se a escala horizontal significa ter múltiplos servidores, como o tráfego dos nossos usuários chega até eles de forma inteligente e organizada?”

A resposta para essa pergunta é um dos componentes críticos de qualquer arquitetura moderna e resiliente: o “LoadBalancer” ou “Balanceador de Carga”. Ele é o verdadeiro “maestro” da orquestra, garantindo que nenhum servidor fique sobrecarregado e que a aplicação continue funcionando mesmo que um dos “músicos” falhe.

Nesta aula, vamos desmistificar completamente esse componente. Você descobrirá que existem diferentes “cérebros” de LoadBalancers, cada um operando em um nível de inteligência distinto. Vamos explorar como eles decidem para qual servidor enviar a próxima requisição e, o mais importante, como eles sabem se um servidor está “saudável” e pronto para trabalhar. Ao final, você entenderá como os LoadBalancers são a peça-chave que permite que empresas como a Netflix e o Spotify lancem novas funcionalidades para milhões de usuários sem que ninguém perceba, por meio de estratégias de deploy com zero tempo de inatividade.

HANDS ON

Nas videoaulas que compõem esta seção, faremos um mergulho prático-conceitual para conectar a teoria dos LoadBalancers com as ferramentas que você encontrará no mercado de trabalho. Nosso objetivo é que você não apenas entenda os diagramas de arquitetura, mas que também se sinta confortável ao visualizar como esses componentes são configurados no mundo real. Por meio do compartilhamento de tela, vamos explorar as interfaces e os arquivos de configuração que dão vida a esses sistemas, solidificando o seu aprendizado.

Primeiramente, faremos uma demonstração conceitual no painel da Amazon Web Services (AWS) para entender a criação de um Application Load Balancer (ALB). Vamos navegar pelas seções de configuração, identificando onde definimos os "Listeners" (as portas de entrada), os "Target Groups" (os grupos de servidores de destino) e, crucialmente, as regras de "Health Check" (as verificações de saúde). Este exercício visual ajudará a materializar o fluxo de configuração de um balanceador de carga em um dos maiores provedores de nuvem do mundo.

Em seguida, nosso foco será no universo dos contêineres com o Kubernetes. Analisaremos um arquivo de configuração simples, no formato YAML, para um objeto Ingress. Explicaremos como o Ingress atua como um roteador inteligente para os serviços dentro de um cluster, direcionando o tráfego externo para as aplicações corretas. Ao final, você terá uma compreensão clara de como os conceitos de balanceamento se aplicam tanto em ambientes de máquinas virtuais tradicionais quanto em orquestradores de contêineres modernos, uma habilidade essencial para qualquer profissional da área.

SAIBA MAIS

Após, na aula anterior, entendermos que a escala horizontal é a base para sistemas resilientes, surge a necessidade de um mecanismo para gerenciar o tráfego destinado a essa frota de servidores. Este mecanismo é o LoadBalancer (LB). Em sua essência, ele é um servidor proxy que se posiciona entre os clientes e os servidores da aplicação, atuando como único ponto de contato e abstração. Sua função primordial é distribuir as requisições recebidas de forma eficiente entre os servidores disponíveis, mas seus benefícios vão além, incluindo a garantia de alta disponibilidade, o aumento da confiabilidade e a simplificação do gerenciamento da infraestrutura. Pense nele como o controlador de tráfego aéreo de um grande aeroporto: ele não apenas direciona os aviões para as pistas disponíveis, mas também garante que as operações sejam seguras, eficientes e contínuas, mesmo que uma das pistas precise ser interditada.

No entanto, nem todos os controladores são iguais. Existem aqueles que operam com base em informações de baixo nível (a rota e o tipo da aeronave) e aqueles que possuem uma visão completa e contextual do destino final de cada passageiro. No mundo dos LBs, essa diferença é definida pela camada do modelo OSI em que eles operam, resultando em **dois tipos principais: Camada 4 (L4) e Camada 7 (L7).**

Um **LoadBalancer de Camada 4** opera na camada de transporte do modelo OSI, lidando diretamente com protocolos como TCP e UDP. Suas decisões de roteamento são baseadas em informações contidas nos primeiros pacotes de uma conexão, como o endereço IP de origem, o endereço IP de destino e as portas envolvidas. Ele não possui visibilidade sobre o conteúdo da comunicação; para ele, uma requisição para buscar uma imagem e uma transação de pagamento são indistinguíveis, são apenas pacotes de dados a serem encaminhados. Por essa simplicidade, os LBs L4 são extremamente rápidos, capazes de lidar com milhões de requisições por segundo com latência mínima. Um exemplo clássico é o Network LoadBalancer (NLB) da AWS, ideal para cenários que exigem altíssima vazão (throughput), como em jogos on-line, streaming de vídeo ou para balancear outros LBs de camada superior.

Já um **LoadBalancer de Camada 7** opera na camada de aplicação, o que significa que ele compreende protocolos como HTTP e HTTPS. Ele é muito mais inteligente porque pode inspecionar o conteúdo de cada requisição. Essa capacidade de "leitura" permite um roteamento muito mais sofisticado e contextual. Por exemplo, um LB L7 pode analisar o caminho da URL e direcionar requisições para **/api/pagamentos** a um grupo de servidores robustos e seguros, enquanto o tráfego para **/blog** é enviado para servidores mais simples. Ele pode ler o cabeçalho **User-Agent** e enviar usuários de dispositivos móveis para uma versão otimizada do site. Pode inspecionar o cabeçalho **Accept-Language** para rotear usuários para servidores que entregam conteúdo em seu idioma nativo, ou ler um cookie para participar de um teste A/B, enviando parte dos usuários para uma nova funcionalidade. O Application Load Balancer (ALB) da AWS é um exemplo perfeito deste tipo, sendo a escolha padrão para a maioria das aplicações web modernas.

Além da camada de operação, os LBs também se diferenciam pelo seu posicionamento na arquitetura. Um LoadBalancer Externo é aquele que possui uma interface pública com a internet, identificado por um endereço IP público ou um nome DNS. Ele é o portão de entrada da sua aplicação, recebendo o tráfego dos usuários e distribuindo-o para a primeira camada de servidores. Por outro lado, um LoadBalancer Interno opera exclusivamente dentro da sua rede privada (VPC), sendo acessível apenas por outros recursos dentro da mesma rede. Sua função é gerenciar a comunicação entre os diferentes serviços da sua aplicação. Em uma arquitetura de microsserviços, por exemplo, é comum ter um LB interno para que o serviço de "Carrinho de Compras" possa se comunicar de forma escalável e resiliente com o serviço de "Estoque", sem expor o serviço de Estoque diretamente à internet, o que também representa um ganho de segurança.

Para decidir para qual servidor enviar a próxima requisição, os LBs utilizam diferentes algoritmos, cada um com suas próprias características. Aprofundar-se neles é crucial para otimizar a distribuição de carga:

Round-robin: o mais simples e comum. Ele mantém uma lista de servidores e envia cada nova requisição para o próximo da lista, em um ciclo contínuo. É justo e previsível, mas não leva em conta a carga atual ou a capacidade de cada servidor. Uma variante, o Weighted Round-robin, permite atribuir "pesos" aos servidores,

fazendo com que servidores mais potentes recebam uma proporção maior de requisições;

Least Connections: é uma estratégia mais inteligente, que direciona a nova requisição para o servidor que, naquele momento, possui o menor número de conexões ativas. É particularmente eficaz em cenários onde as conexões têm durações muito diferentes, como em uploads de arquivos ou sessões de streaming, pois evita que um servidor fique "preso" com muitas conexões longas enquanto outros estão ociosos;

IP Hashing: neste método, o LB calcula um "hash" a partir do endereço IP do cliente e o utiliza para determinar para qual servidor a requisição será enviada. Isso garante que requisições do mesmo IP sempre caiam no mesmo servidor. Embora útil para aplicações legadas que precisam de "afinidade de sessão" (session affinity), essa abordagem tem desvantagens: pode levar a uma distribuição de carga desigual se muitos usuários estiverem atrás de um mesmo proxy (como em uma rede corporativa) e pode causar problemas de sessão se um servidor do grupo falhar, pois o cálculo do hash pode mudar e redirecionar os usuários.

Contudo, distribuir o tráfego de forma inteligente é inútil se os destinos não forem confiáveis. É aqui que entra o conceito vital de Health Checks (Verificações de Saúde). O LB atua como um supervisor vigilante, testando ativamente a saúde de cada servidor em seu grupo. Essas verificações são altamente configuráveis, com parâmetros como o **intervalo** (a frequência dos testes), o **timeout** (o tempo de espera por uma resposta), o **unhealthy threshold** (o número de falhas consecutivas para marcar um servidor como "não saudável") e o **healthy threshold** (o número de sucessos consecutivos para trazê-lo de volta). Se um servidor falha nos testes, o LB executa um processo de failover: ele para de enviar tráfego novo para a instância doente de forma imediata e transparente para o usuário. LBs mais avançados também implementam "connection draining", permitindo que as conexões já existentes naquele servidor terminem naturalmente antes de removê-lo completamente.

A capacidade de roteamento avançado dos LBs L7 abre portas para estratégias de deploy que são a base do DevOps moderno e da entrega contínua. O Blue/Green Deployment é uma técnica poderosa para eliminar o tempo de inatividade durante as atualizações. Mantêm-se dois ambientes de produção

idênticos e isolados: **"Blue" (a versão estável atual)** e **"Green" (a nova versão)**. O LB direciona 100% do tráfego para o ambiente Blue. Quando a nova versão no ambiente Green está pronta e validada, a equipe de operações realiza a "virada": o LB é reconfigurado para apontar todo o tráfego para o ambiente Green. O deploy é instantâneo. Se for detectado qualquer problema, a reversão é igualmente rápida, bastando apontar o LB de volta para o ambiente Blue. Essa estratégia, no entanto, exige atenção especial ao banco de dados, que geralmente é compartilhado. As alterações de schema devem ser retrocompatíveis para que ambas as versões da aplicação possam coexistir.

Uma estratégia ainda mais sofisticada e segura é o **Canary Deployment**. O nome vem da antiga prática de mineiros que levavam canários para as minas de carvão; se gases tóxicos estivessem presentes, o pássaro, mais sensível, sentiria primeiro. No desenvolvimento de software, a nova versão é o "canário". O LB é configurado para desviar uma pequena porcentagem do tráfego (1%, 5%) para a nova versão, enquanto a maioria dos usuários continua na versão estável. A equipe então monitora intensivamente o canário, observando não apenas métricas técnicas (taxa de erros, latência), mas também métricas de negócio (taxa de conversão, engajamento). Se o canário se mostrar saudável e performático, o tráfego é gradualmente aumentado até que a nova versão receba 100% dos usuários e se torne a versão estável. Isso permite lançar novas funcionalidades de forma controlada e com risco extremamente baixo.

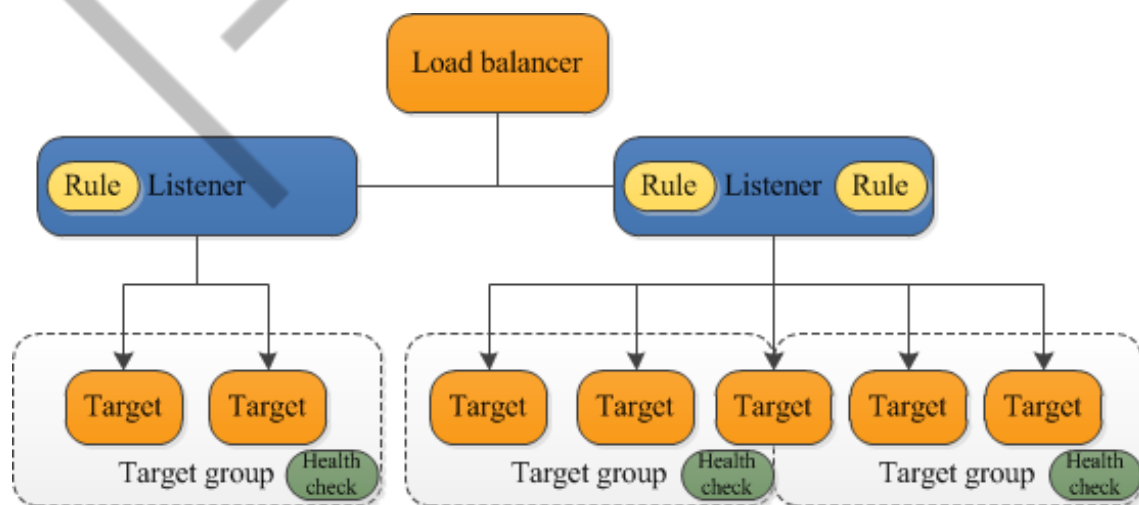


Figura 1 - LoadBalancer AWS
Fonte: docs.aws.amazon.com (s.d)

MERCADO, CASES E TENDÊNCIAS

O balanceamento de carga é uma disciplina consolidada, mas que continua evoluindo rapidamente com o avanço da computação em nuvem, DevOps e arquiteturas de microsserviços. Observar o mercado nos mostra que o LoadBalancer moderno é muito mais do que um simples distribuidor de tráfego.

Case: A Batalha da Netflix contra Falhas em Cascata

A Netflix é famosa por sua cultura de resiliência e por popularizar a "Engenharia do Caos". Um dos pilares para que eles possam desligar serviços intencionalmente sem derrubar a plataforma é o uso sofisticado de balanceadores de carga e roteamento de tráfego. Eles utilizam LBs internos para desacoplar seus milhares de microsserviços. Se um serviço crítico, como o de recomendações, começa a apresentar lentidão, o sistema de balanceamento pode automaticamente redirecionar o tráfego para uma versão de "fallback" (uma versão mais simples do serviço) ou ativar "circuitbreakers", impedindo que a falha em um componente se espalhe e cause uma falha em cascata em todo o sistema.

Tendência: A Evolução para Service Mesh e Gateways de API

No ecossistema de Kubernetes e microsserviços, o conceito de balanceamento de carga evoluiu. Ferramentas como Istio e Linkerd criam uma "malha de serviços" (Service Mesh) que gerencia a comunicação entre todos os serviços. Elas utilizam "sidecar proxies", que atuam como LBs L7 ultra-avançados para cada serviço, oferecendo observabilidade detalhada, segurança com criptografia mútua (mTLS) e controle de tráfego extremamente granular. Essa tendência move a inteligência do balanceamento de carga para mais perto da aplicação, permitindo um controle sem precedentes.

Notícia: A Ascensão do Balanceamento de Carga Multi-Cloud e Global (GSLB)

Com a crescente adoção de estratégias multi-cloud (usar mais de um provedor de nuvem, como AWS e Azure) e híbridas (nuvem e on-premise), as empresas enfrentam o desafio de direcionar seus usuários para o melhor data center disponível. O Global Server LoadBalancing (GSLB) é a tecnologia que resolve isso. Usando o DNS de forma inteligente, o GSLB pode direcionar um usuário do Brasil

para o data center em São Paulo e um usuário da Europa para o data center em Frankfurt, com base em geolocalização, latência e saúde dos ambientes. Empresas como a Cloudflare e a F5 são líderes nesse mercado, garantindo performance e resiliência em escala global.

EMANIP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, desvendamos um dos componentes essenciais de qualquer arquitetura escalável: o LoadBalancer. Você aprendeu que ele atua como o "controlador de tráfego" do seu sistema, distribuindo requisições para garantir a disponibilidade e evitar a sobrecarga dos servidores. Vimos a diferença entre os balanceadores de Camada 4, que operam com base em informações de rede para máxima velocidade, e os de Camada 7, que entendem o conteúdo da aplicação para permitir um roteamento inteligente.

Exploramos como os algoritmos de balanceamento tomam decisões e como os Health Checks e o processo de failover são vitais para a resiliência do sistema. Por fim, conectamos a teoria à prática do DevOps, descobrindo como a inteligência dos LBs L7 é a tecnologia que possibilita estratégias de deploy avançadas, como Blue/Green e Canary, permitindo que as empresas inovem de forma contínua e segura, sem impactar a experiência do usuário.

REFERÊNCIAS

AWS. **What is an Application Load Balancer?**. Amazon Web Services Documentation. Disponível em: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>. Acesso em: 26 ago. 2025.

KUBERNETES. **Ingress**. Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/ingress/>. Acesso em: 26 ago. 2025.

THE CLOUDFLARE BLOG. **Load Balancing**. Disponível em: <https://www.cloudflare.com/application-services/products/load-balancing/>. Acesso em: 15 ago. 2025.

PALAVRAS-CHAVE

Escalabilidade. Escala Vertical. Escala Horizontal. Autoscaling. Elasticidade.

EMSE



POSTECH